



Προγραμματιστική εργασία: Σημασιολογική ανάκτηση

Χρήστος Ρομπόκος, 3220171

Φάση 2 – Σημασιολογική ανάκτηση χρησιμοποιώντας Transformers και FAISS

Η ακόλουθη αναφορά συνοψίζει την ανάπτυξη και αξιολόγηση ενός συστήματος ανάκτησης πληροφοριών για τη συλλογή IR2025. Ενώ στην πρώτη φάση της εργασίας προσεγγίσαμε αυτό το πρόβλημα με κλασική ανάκτηση με την χρήση του πιθανοτικού μοντέλου BM25 και την μηχανή αναζήτησης Elasticsearch, σε αυτήν την φάση κάνουμε σημασιολογική ανάκτηση με την χρήση Transformers και ευρετηρίου FAISS.

Η μετάβαση στη σημασιολογική αναζήτηση απαιτούσε τη μετάβαση από ένα διακριτό ευρετήριο λέξεων σε έναν συνεχή διανυσματικό χώρο. Για να μετατραπούν τα κείμενα της συλλογής και τα ερωτήματα σε πυκνά διαστήματα – ενθέσεις, χρησιμοποιήθηκε το προεκπαιδευμένο μοντέλο της βιβλιοθήκης transformers, all-MiniLM-L6-v2, ένα γρήγορο και μικρό μοντέλο, το οποίο αντιστοιχίζει προτάσεις σε έναν πυκνό διανυσματικό χώρο 384 διαστάσεων. Για την ευρετηρίαση των 18.316 κειμένων χρησιμοποιήθηκε η συνημιτονοειδής ομοιότητα (cosine similarity) για να ανακτώνται τα κείμενα με βάση την θεματολογία τους και όχι το μέγεθός τους.

Η αξιολόγηση του συστήματος πραγματοποιήθηκε χρησιμοποιώντας το εργαλείο trec_eval για να συγκριθούν τα ανακτημένα αποτελέσματα με το δοθέν αρχείο qrels.txt σε τρία βάθη ανάκτησης ($k=20, 30, 50$). Για την εκτέλεση του results_k20, το σύστημα πέτυχε Mean Average Precision (MAP) 0.3129, με Precision@5 0.6600 και Precision@20 0.3500. Όταν το βάθος ανάκτησης αυξήθηκε στο results_k30, το MAP αυξήθηκε στο 0.3413, ενώ οι τιμές Precision@k παρέμειναν σταθερές. Η τελική αξιολόγηση στο results_k50 έφτασε στο υψηλότερο MAP, 0.3744. Αυτή η τάση δείχνει ότι, ενώ η κατάταξη των κορυφαίων αποτελεσμάτων παραμένει σταθερή, η αύξηση του αριθμού των ανακτημένων εγγράφων βελτιώνει σημαντικά την ανάκληση, καταγράφοντας σχετικά κείμενα που τοποθετήθηκαν χαμηλότερα στη λίστα κατάταξης.

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.26200.7623]
(c) Microsoft Corporation. All rights reserved.

C:\Users\30695\Desktop\4oETOS\7o6MHNO\Systhmata_Anakthshs\ergasia2\ToRun>trec_eval.exe -m map -m P.5,10,15,20 qrels.txt resul
ts_k20.txt
1 [main] trec_eval 19860 find_fast_cwd: WARNING: Couldn't compute FAST_CWD pointer. Please report this problem to
the public mailing list cygwin@cygwin.com
map      all      0.3129
P_5      all      0.6600
P_10     all      0.5000
P_15     all      0.3933
P_20     all      0.3500

C:\Users\30695\Desktop\4oETOS\7o6MHNO\Systhmata_Anakthshs\ergasia2\ToRun>trec_eval.exe -m map -m P.5,10,15,20 qrels.txt resul
ts_k30.txt
3 [main] trec_eval 15648 find_fast_cwd: WARNING: Couldn't compute FAST_CWD pointer. Please report this problem to
the public mailing list cygwin@cygwin.com
map      all      0.3413
P_5      all      0.6600
P_10     all      0.5000
P_15     all      0.3933
P_20     all      0.3500

C:\Users\30695\Desktop\4oETOS\7o6MHNO\Systhmata_Anakthshs\ergasia2\ToRun>trec_eval.exe -m map -m P.5,10,15,20 qrels.txt resul
ts_k50.txt
3 [main] trec_eval 8088 find_fast_cwd: WARNING: Couldn't compute FAST_CWD pointer. Please report this problem to
the public mailing list cygwin@cygwin.com
map      all      0.3744
P_5      all      0.6600
P_10     all      0.5000
P_15     all      0.3933
P_20     all      0.3500
```

Παρατηρώντας τα αποτελέσματα της αξιολόγησης από ερώτημα προς ερώτημα, παρατηρείται ότι το μοντέλο δεν είναι εξίσου αποδοτικό σε κάθε ερώτημα. Για παράδειγμα, το ερώτημα Q02 έχει MAP 0.1572, ενώ το ερώτημα Q09 έχει MAP 0.6670. Όμως, φαίνεται πως υπάρχει η τάση να κατατάσσονται στις πρώτες θέσεις τα πιο σχετικά κείμενα, καθώς η ακρίβεια στα 5 πρώτα ανακτηθέντα κείμενα είναι μεγαλύτερη από την μέση ακρίβεια για κάθε ένα ερώτημα.

```
C:\Users\30695\Desktop\4oETOS\7o6MHNO\Systhmata_Anakthshs\ergasia2\ToRun>trec_eval.exe -q -m map -m P.5 qrels.txt res
ults_k50.txt
3 [main] trec_eval 1084 find_fast_cwd: WARNING: Couldn't compute FAST_CWD pointer. Please report this problem
to
the public mailing list cygwin@cygwin.com
map      Q01      0.2634
P_5      Q01      0.4000
map      Q02      0.1572
P_5      Q02      0.4000
map      Q03      0.3756
P_5      Q03      0.6000
map      Q04      0.4672
P_5      Q04      0.8000
map      Q05      0.3886
P_5      Q05      0.6000
map      Q06      0.4476
P_5      Q06      1.0000
map      Q07      0.4074
P_5      Q07      0.6000
map      Q08      0.2570
P_5      Q08      0.6000
map      Q09      0.6670
P_5      Q09      1.0000
map      Q10      0.3133
P_5      Q10      0.6000
map      all      0.3744
P_5      all      0.6600
```

Η υλοποίηση της φάσης 2 είχε χαμηλότερο MAP σε όλα τα επίπεδα σε σχέση με την υλοποίηση της φάσης 1. Συγκεκριμένα, η κλασική ανάκτηση με τον BM25 είχε MAP 0.5685, ενώ η σημασιολογική ανάκτηση με Transformers και FAISS είχε 0.3744. Αυτό συμβαίνει γιατί ο BM25 βασίζεται στην αντιστοιχία λέξεων ανάμεσα σε ερωτήματα και κείμενα, λαμβάνει υπ' όψιν την συχνότητα εμφάνισης των όρων και το μέγεθος των κειμένων, σε αντίθεση με τις πυκνές διανυσματικές αναπαραστάσεις, οι οποίες βασίζονται στην σημασιολογία των κειμένων. Αυτό μπορεί να έχει ως αποτέλεσμα την ανάκτηση κειμένων με συναφή θεματολογία, τα οποία όμως να μην ανταποκρίνονται στο ερώτημά μας. Επιπροσθέτως, η συλλογή μας είναι σχετικά μικρή, οπότε φαίνεται να έχει μεγαλύτερη σημασία η αντιστοιχία λέξεων-κλειδιά, παρά η θεματολογία του κειμένου.

Κατά τη φάση ανάπτυξης, διεξήχθη μια πειραματική εκτέλεση χρησιμοποιώντας το “μεγαλύτερο” μοντέλο 768 διαστάσεων, all-mpnet-base-v2. Το μοντέλο αυτό, όμως, αποδείχθηκε λιγότερο αποδοτικό, τόσο ως προς την ακρίβεια, όσο και ως προς τον χρόνο εκτέλεσης. Πιθανότατα αυτό το μοντέλο έγινε υπερβολικά αναλυτικό και εντόπισε σχέσεις μεταξύ άσχετων κειμένων.

```
C:\Users\30695\Desktop\4oETOS\7o6MHNO\Systhmata_Anakthshs\ergasia2\ToRun>trec_eval.exe --m map -m P.5,10,15,20 qrels.txt alt_results_k50.txt
3 [main] trec_eval 16528 find_fast_cwd: WARNING: Couldn't compute FAST_CWD pointer. Please report this problem
to
the public mailing list cygwin@cygwin.com
map      all      0.3683
P_5      all      0.5800
P_10     all      0.4200
P_15     all      0.3867
P_20     all      0.3500
```

Το σημασιολογικό σύστημα ανάκτησης αποδείχθηκε ένα ισχυρό εργαλείο για την ανάκτηση πληροφοριών με βάση το νόημα των κειμένων, ιδιαίτερα για ερωτήματα όπου ο χρήστης μπορεί να μην γνωρίζει τους ακριβείς όρους. Ωστόσο, τα αποτελέσματα του BM25 της φάσης 1 παρήγαγαν ανώτερα αποτελέσματα για την συγκεκριμένη συλλογή κειμένων.

Επιπλέον screenshots:

Screenshot από την εκτέλεση του κώδικα:

```
C:\Users\30695\Desktop\4oETOS\7o6MHNO\Systhmata_Anakthshs\ergasia2\ToRun>py phase2_solution.py
Step 1: Loading documents and queries...
Loaded 18316 documents and 10 queries.
Step 2: Loading existing embeddings from disk...
Embeddings loaded successfully.
Step 3: Building FAISS index...
Index built with 18316 vectors.
Step 4: Running searches and generating results...
Created results_k20.txt
Created results_k30.txt
Created results_k50.txt

--- All Steps Complete! ---
```

Screenshot της εναλλακτικής υλοποίησης:

```
phase2_alt.py
1
2 import pandas as pd
3 import numpy as np
4 import faiss
5 import os
6 import pickle
7 from sentence_transformers import SentenceTransformer
8
9 # STEP 1: Load and Preprocess Data
10 print("Step 1: Loading documents and queries...")
11 df = pd.read_csv('documents.csv')
12 queries_df = pd.read_csv('queries.csv')
13
14 # Preprocessing
15 df['prepared_text'] = df['Text'].astype(str).str.strip()
16 doc_ids = df['ID'].tolist()
17
18 # Prepare queries
19 queries_df['prepared_text'] = queries_df['Text'].astype(str).str.strip()
20 query_dict = dict(zip(queries_df['ID'], queries_df['prepared_text']))
21
22 print(f"Loaded {len(df)} documents and {len(query_dict)} queries.")
23
24 # STEP 2: Generate or Load Embeddings
25 # Using all-mpnet-base-v2 (768 dimensions)
26 model = SentenceTransformer('all-mpnet-base-v2')
27 embedding_file = 'doc_embeddings_mpnet_2.pkl'
28
29 if not os.path.exists(embedding_file):
30     print("Step 2: Generating document embeddings (this may take a few minutes)...")
31     doc_embeddings = model.encode(df['prepared_text'].tolist(), show_progress_bar=True, convert_to_numpy=True)
32
33     with open(embedding_file, 'wb') as f:
34         pickle.dump({'ids': doc_ids, 'embeddings': doc_embeddings}, f)
35     print("Embeddings generated and saved.")
36 else:
37     print("Step 2: Loading existing embeddings from disk...")
38     with open(embedding_file, 'rb') as f:
39         data = pickle.load(f)
40     doc_embeddings = data['embeddings']
41     print("Embeddings loaded successfully.")
42
```

```

phase2_alt.py
40     doc_embeddings = data['embeddings']
41     print("Embeddings loaded successfully.")
42
43     # STEP 3: Build the FAISS Index
44     print("Step 3: Building FAISS index...")
45     dimension = doc_embeddings.shape[1]
46
47     # IndexFlatIP + L2 Normalization = Cosine Similarity
48     index = faiss.IndexFlatIP(dimension)
49     faiss.normalize_L2(doc_embeddings)
50     index.add(doc_embeddings)
51
52     print(f"Index built with {index.ntotal} vectors.")
53
54     # STEP 4: Semantic Search
55     print("Step 4: Running searches and generating results...")
56     k_values = [20, 30, 50]
57     tag = "mpnet_faiss_cosine"
58
59     for k in k_values:
60         output_filename = f"alt_results_k{k}.txt"
61         with open(output_filename, 'w') as f:
62             for q_id, q_text in query_dict.items():
63                 # a) Encode and Normalize Query
64                 query_vec = model.encode([q_text], convert_to_numpy=True)
65                 faiss.normalize_L2(query_vec)
66
67                 # b) Search FAISS
68                 D, I = index.search(query_vec, k)
69
70                 # c) Write in TREC format: query_id Q0 doc_id rank score tag
71                 for rank, (doc_idx, score) in enumerate(zip(I[0], D[0])):
72                     doc_id = doc_ids[doc_idx]
73                     # Format: query_id Q0 doc_id rank score tag
74                     line = f"{q_id} Q0 {doc_id} {rank + 1} {score:.6f} {tag}\n"
75                     f.write(line)
76
77     print(f"Created {output_filename}")
78
79     print("\n--- All Steps Complete! ---")
80

```

Ως πηγές χρησιμοποιήθηκαν οι διαφάνειες του μαθήματος και το βιβλίο του Manning.